Métodos especiais e Classe Estática

Além do __init__: customizando comportamentos e organizando responsabilidades

O que são métodos especiais?

Python reserva uma série de métodos com nome cercados por dois *underscores* de cada lado

Eles são chamados de ***dunder methods***

Quem chama esses dunder methods?

Assim como o __init__, esses métodos são chamados automaticamente, pelo Python, em situações específicas

- imprimindo um objeto
- medindo comprimento
- usando operadores
- etc

Eles permitem que seus objetos se comportem como **objetos nativos do python**

Conhecendo cada um dos dunder methods

LIVRO DOS MÉTODOS MÁGICOS (SPECIAL METHODS) EM PYTHON

__init__

NA CRIAÇÃO DO OBJETO

```
obj = MinhaClasse()
```

Inicializa o objeto, define atributos iniciais. O "construtor".

__str__

EM print(obj) e str(obj)

```
print(meu_livro)
```

Retorna uma representação legível para humanos. Para usuários final.

__repr__

NO CONSOLE INTERATIVO E repr(obj)

```
>> meu_livro
```

Retorna uma representação técnica/debug. Para desenvolvedores.

__len__

EM len(obj)

```
tamanho = len(minha_lista)
```

Retorna o tamanho ou número de itens de um objeto.

__eq__

NO OPERADOR ==

```
obj1 == obj2
```

Compara igualdade entre dois objetos com base em atributos.

__lt__

NO OPERADOR <

```
preço < 100
```

Define a ordenação "menor que" para o objeto.

__add__

NO OPERADOR +

```
ponto_a + ponto_b
```

Define a operação de soma para o objeto.

__contains__

NO OPERADOR in

```
'python' in cursos
```

Verifica se um item pertence ao objeto.

UTILIZE MÉTODOS MÁGICOS PARA TORNAR SEUS OBJETOS MAIS PODEROSOS

Exemplos

str X repr

STR

Para o usuário final

deve ser legível

REPR

Para o desenvolvedor

Deve ser preciso, idealmente permitindo recriar o objeto

Caso apenas um seja definido, Python usa o __repr__ como fallback

Dúvidas?

Métodos de Classe

O que é

Método de classe pertence à classe em si, não a um objeto

Ao invés de receber *self*, que é a referência ao objeto, ele recebe *cls*, que é a referência à classe

Decorado com

@classmethod

Quando usar

Criação de construtores personalizados

Mexer com atributos de classe

Um formulário de cadastro que pode ser preenchido de dois jeitos:

- Digitando todos os dados manualmente
- Importando um arquivo CSV (excel)

O resultado final é o mesmo, mas o caminho é diferente

Exemplos

Cadastro de usuários

```python
# cadastro manual
usuario1 = Usuario("Carlos", "carlos@email.com", 28)
print(usuario1)


#cadastro a partir de um arquivo CSV
usuarios = Usuario.from_csv("usuarios.csv")
```

Métodos estáticos

O que são

Função que fica dentro de uma classe, mas não depende de nada dela

É apenas uma função organizada dentro de uma classe

Como declarar

@staticmethod

```
class Calculadora:

    @staticmethod
    def somar(a, b):
        return a + b
```

Por que e quando usar

- Função tem relação com a classe, mas não precisa dela
  - Validação
  - Cálculo
  - Fromatação
- Você quer organizar o código e não deixar funções soltas
-

Quando não usar

- Precisa acessar atributos
  - usa o self
- Acessa ou modifica a classe
  - Usa classmethod

Exemplo

- Validar email de usuário
- Validar CPF
- Cotação de moeda

Dúvidas?

Quando usar cada tipo

Instância, classe e estático

COMPARATIVO VISUAL: TIPOS DE MÉTODOS EM PYTHON

TIPO DE MÉTODO	DECORATOR	PRIMEIRO PARÂMETRO	ACESSA INSTÂNCIA?	ACESSA CLASSE?	USO PRINCIPAL
self INSTÂNCIA	(nenhum)	self	✅ Sim	⚠ Indiretamente	TRABALHAR COM DADOS DO OBJETO
C cls CLASSE	@classmethod	cls	❌ Não	✅ Sim	CONSTRUTORES ALTERNATIVOS / CONFIG
ESTÁTICO	@staticmethod	(nenhum)	❌ Não	❌ Não	FUNÇÕES UTILITÁRIAS

Regra para se pensar ao criar método

- Precisa acessar os dados do objeto?
    - instância, usando self
- Precisa acessar os dados da classe?
    - classe, usando cls
    - @classmethod
- Não precisa de nada?
    - estático
    - @staticmethod

Dúvidas?

Recapitulando

- Métodos especiais
- Métodos de classe
- Métodos estáticos